Metody obliczeniowe w nauce i technice

Rozwiązywanie równań liniowych z macierzą trójdiagonalną za pomocą algorytmu Gaussa i algorytmu Thomasa.

Bartłomiej Prus

Gr. Śr. 11:30

Dane techniczne urządzenia, na którym wykonano ćwiczenie:

System: Windows 11 x64

Procesor: Intel Core I9 @2.30 GHz 11 generacji

Pamięć RAM: 16GB

Środowisko: jupyter notebook

Ćwiczenie zostało zrealizowane w języku Python 3, przy użyciu bibliotek numpy (do operacji numerycznych)), matplotlib

Opis realizacji ćwiczenia

Ćwiczenie polegało na wykonaniu algorytmu Gaussa i Thomasa na zadanym układzie z macierzą trójdiagonalną. Następnie porównanie czasu działania algorytmów, jak i błędów otrzymanych poprzez użycie tych algorytmów. Rozmiary układu były badane od trzech do tysiąca, a w dokumencie poniżej zostały dodane jedynie wybrane wartości.

Badane rzeczywiste x zostało wygenerowane jako jedna tablica, z losową permutacją {-1, 1} o rozmiarze 1000, gdzie do mniejszych rozmiarów układów, były brane elementy od 0 do wielkości układu.

1. Badany układ z macierzą trójdiagonalną

$$\begin{cases} a_{i,i} = 4 \\ a_{i,i+1} = \dfrac{1}{i+5} \\ a_{i,i-1} = \dfrac{4}{i+6}\ dla\ i > 1 \\ a_{i,i-1} = 0\ dla\ j < i-1\ oraz\ j > i+1 \end{cases} \qquad \boldsymbol{i, j = 1, \dots, n}$$

2. Macierz wyrazów wolnych

Macierz wyrazu wolnego została obliczona, za pomocą wymnożenia badanej macierzy i oczekiwanej macierzy wynikowej.

3. Sposób przechowywania macierzy w przypadku obu algorytmów

W przypadku macierzy trójdiagonalnych, algorytm Thomasa wykorzystuje jedynie 3 przekątne macierzy, i jedynie one zostały zapisane i są przechowywane w 3 osobnych wektorach, w celu zminimalizowania zużycia pamięci, gdyż algorytm ten nie wykorzystuje pozostałych zer, którymi wypełniona jest macierz.

W przypadku algorytmu Gaussa, musimy jednak przechowywać całą macierz wraz z inicjalnymi zerami, co skutkuje znacznie większym obciążeniem pamięci, ponieważ wymaga tego sposób działania algorytmu Gaussa.

4. Badane własności układów z wykorzystaniem macierzy 1 i 2 z użyciem algorytmu Gaussa

4.1. Błąd maksymalny

Poszczególne pozycje macierzy wynikowej są porównywane ze znanymi rozwiązaniami za pomocą wzoru:

$$\max\left(\left|x_{wynikowe} - x_{realne}\right|\right)$$

4.2. Precyzja obliczeń

Oba układy są badane w pojedynczej i podwójnej precyzji

4.3. Czas działania algorytmu

Funkcja perf_counter z modułu time

4.4. Pamięć zajmowana przez struktury algorytmu

Atrybut .nbytes dla tablic numpyowych

romiar układu	Eliminacja Gaussa			Algorytm Thomasa		
	błąd maksymalny	czas działania [s]	potrzebna pamięć [bajty]	błąd maksymalny	czas działania [s]	potrzebna pamięć [bajty]
2	1,19209E-07	0,00149	16	1,19E-07	0,00048	16
3	5,96046E-08	0,00035	36	1,19E-07	1,99E-05	28
4	5,96046E-08	0,00008	64	1,19E-07	1,87E-05	40
5	1,19209E-07	0,00009	100	1,19E-07	2,60E-05	52
6	1,19209E-07	0,00008	144	1,19E-07	3,20E-05	64
7	1,19209E-07	0,00010	196	1,19E-07	2,07E-05	76
8	1,19209E-07	0,00015	256	1,19E-07	2,17E-05	88
9	1,19209E-07	0,00014	324	1,19E-07	1,19E-05	100
10	1,19209E-07	0,00016	400	1,19E-07	9,86E-06	112
11	1,19209E-07	0,00020	484	1,19E-07	1,07E-05	124
12	1,19209E-07	0,00023	576	1,19E-07	1,09E-05	136
13	1,19209E-07	0,00025	676	1,19E-07	1,24E-05	148
14	1,19209E-07	0,00031	784	1,19E-07	1,22E-05	160
15	1,19209E-07	0,00033	900	1,19E-07	1,35E-05	172
16	1,19209E-07	0,00049	1024	1,19E-07	1,40E-05	184
17	1,19209E-07	0,00084	1156	1,19E-07	1,63E-05	196
18	1,19209E-07	0,00046	1296	1,79E-07	1,62E-05	208
19	1,19209E-07	0,00050	1444	1,79E-07	1,84E-05	220
20	1,19209E-07	0,00053	1600	1,79E-07	1,70E-05	232
30	1,19209E-07	0,00106	3600	1,79E-07	3,79E-05	352
40	1,19209E-07	0,00276	6400	1,79E-07	4,41E-05	472
50	1,19209E-07	0,00324	10000	2,38E-07	0,00011	592
100	1,19209E-07	0,01060	40000	2,38E-07	8,49E-05	1192
200	1,19209E-07	0,04193	160000	2,38E-07	0,00016	2392
300	1,19209E-07	0,08655	360000	2,38E-07	0,00027	3592
400	1,19209E-07	0,15896	640000	2,38E-07	0,00033	4792
500	1,19209E-07	0,24058	1000000	2,38E-07	0,00055	5992
600	1,19209E-07	0,35728	1440000	2,38E-07	0,00062	7192
700	1,19209E-07	0,49979	1960000	2,38E-07	0,00073	8392
800	1,19209E-07	0,66055	2560000	2,38E-07	0,00067	9592
900	1,19209E-07	0,84252	3240000	2,38E-07	0,00076	10792
1000	1,19209E-07	1,03504	4000000	2,38E-07	0,00085	11992

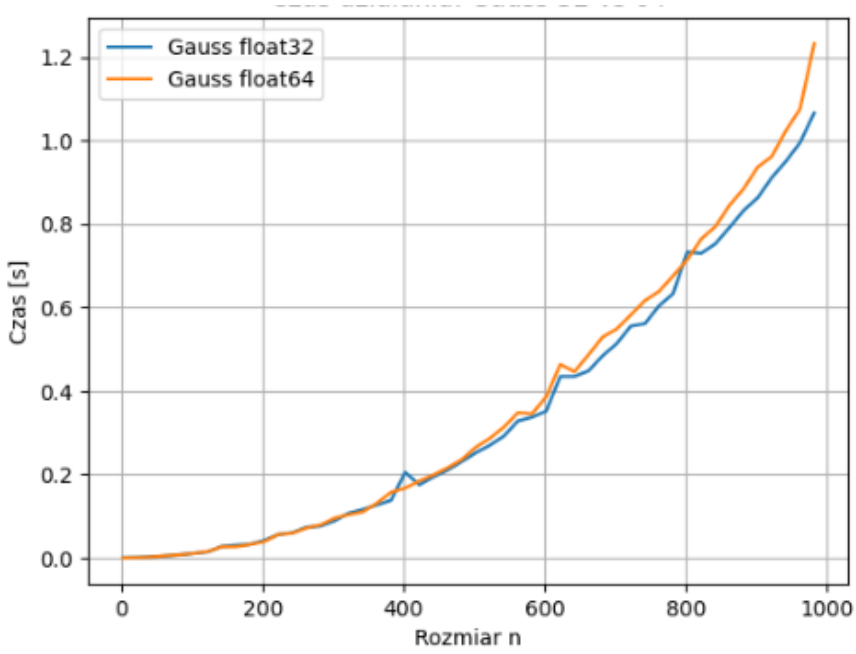
Tabela 1: Badane wartości układu dla obu badanych algorytmów przy pojedynczej precyzji

romiar układu	Eliminacja Gaussa			Algorytm Thomasa		
	błąd maksymalny	czas działania [s]	potrzebna pamięć [bajty]	błąd maksymalny	czas działania [s]	potrzebna pamięć [bajty]
2	1,11E-16	0,00013	32	1,11E-16	0,00025	32
3	0	8,7933E-05	72	2,22E-16	1,81E-05	56
4	2,22E-16	4,9508E-05	128	2,22E-16	0,00019	80
5	2,22E-16	6,2878E-05	200	2,22E-16	3,98E-05	104
6	2,22E-16	7,7536E-05	288	2,22E-16	2,01E-05	128
7	2,22E-16	9,5693E-05	392	2,22E-16	3,08E-05	152
8	2,22E-16	0,00015	512	2,22E-16	2,21E-05	176
9	2,22E-16	0,00016	648	2,22E-16	9,74E-06	200
10	2,22E-16	0,00018	800	2,22E-16	1,02E-05	224
11	2,22E-16	0,00026	968	2,22E-16	1,02E-05	248
12	2,22E-16	0,00026	1152	2,22E-16	3,24E-05	272
13	2,22E-16	0,00028	1352	2,22E-16	1,19E-05	296
14	2,22E-16	0,00029	1568	2,22E-16	2,36E-05	320
15	2,22E-16	0,00031	1800	2,22E-16	1,36E-05	344
16	2,22E-16	0,00041	2048	2,22E-16	1,39E-05	368
17	2,22E-16	0,00042	2312	2,22E-16	2,76E-05	392
18	2,22E-16	0,00041	2592	2,22E-16	1,55E-05	416
19	2,22E-16	0,00051	2888	2,22E-16	2,71E-05	440
20	2,22E-16	0,00052	3200	2,22E-16	2,84E-05	464
30	2,22E-16	0,00109	7200	2,22E-16	7,58E-05	704
40	2,22E-16	0,00192	12800	2,22E-16	4,27E-05	944
50	2,22E-16	0,00289	20000	2,22E-16	8,60E-05	1184
100	2,22E-16	0,01042	80000	2,22E-16	9,63E-05	2384
200	3,33E-16	0,04497	320000	3,33E-16	0,00016	4784
300	3,33E-16	0,08630	720000	3,33E-16	0,00025	7184
400	3,33E-16	0,15389	1280000	3,33E-16	0,00041	9584
500	4,44E-16	0,25028	2000000	4,44E-16	0,00044	11984
600	4,44E-16	0,36127	2880000	4,44E-16	0,00066	14384
700	4,44E-16	0,51750	3920000	4,44E-16	0,00061	16784
800	4,44E-16	0,66618	5120000	4,44E-16	0,00069	19184
900	4,44E-16	0,83806	6480000	4,44E-16	0,00073	21584
1000	4,44E-16	1,07965	8000000	4,44E-16	0,00083	23984

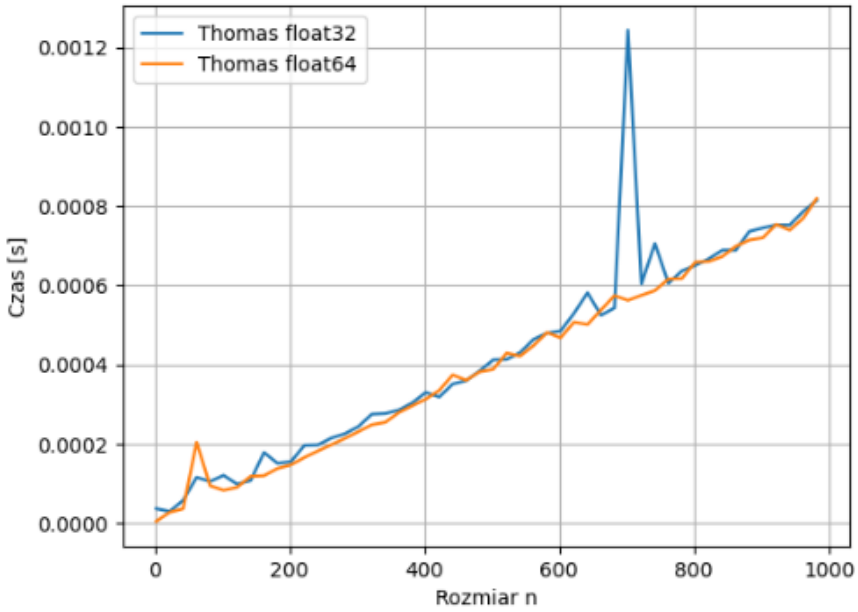
Tabela 2: Badane wartości układu dla obu badanych algorytmów przy podwójnej precyzji

Można zobaczyć, że zajmowana pamięć, jest spodziewana co do obu algorytmów oraz czas również. Z uwagi że algorytm Thomasa ma złożoność liniową, a algorytm Gaussa $n^{3,}$ co widać w znaczących różnicach miedzy nimi.

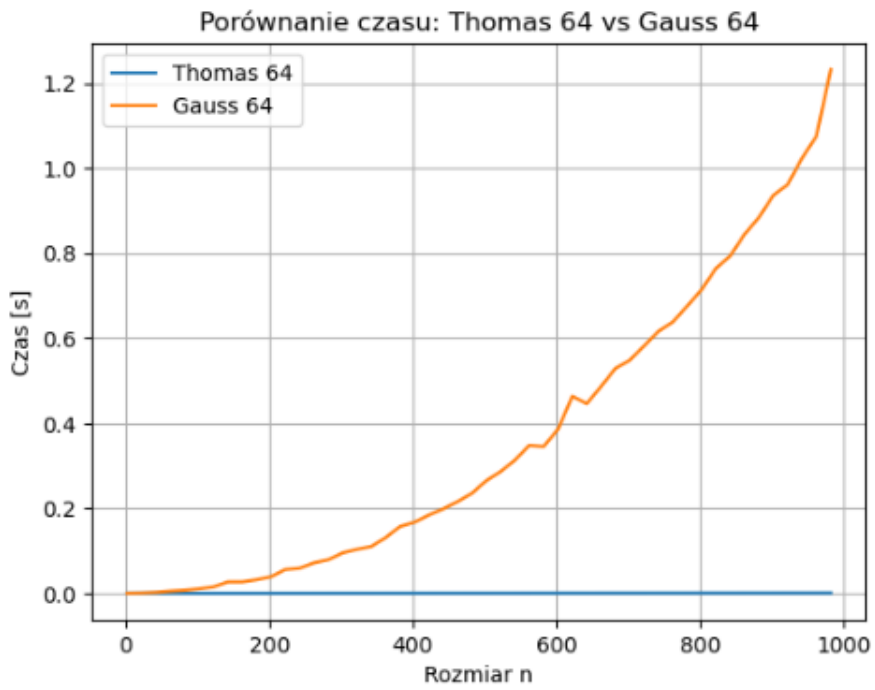
5. Wizualizacja badanych wartości



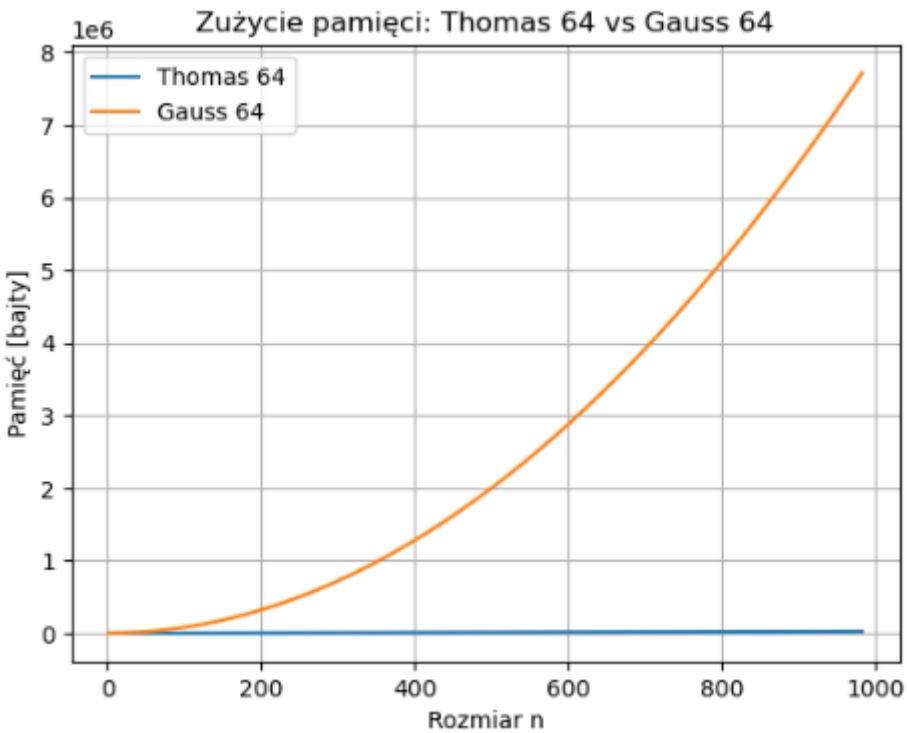
Rysunek 1: Czas działania miedzy różnymi precyzjami algorytmu Gaussa



Rysunek 2: Czas działania miedzy różnymi precyzjami algorytmu Gaussa



Rysunek 3: Porównanie czasu działania obu algorytmów w podwójnej precyzji



Rysunek 4: Porównanie zużycia pamięci w przypadku obu algorytmów

Jak możemy zobaczyć, algorytm Thomasa, rzeczywiście działa w czasie liniowym (rysunek 2,3), a algorytm Gaussa, w czasie n^3 (rysunek 1, 3). Dotyczy to również zajmowanej pamięci przez oba algorytmy (rysunek 4). Co zgadza się z danymi z tabel 1 i 2

6. Wnioski

Algorytm Thomasa jest znacznie lepszy od algorytmu eliminacja Gaussa, jeżeli można go użyć. Thomas działa jedynie dla trójdiagonalnych macierzy, gdyż jest to tak naprawdę algorytm Gaussa dla jednego konkretnego przypadku. Więc algorytm Thomasa jest lepszy pod każdym względem, ale jest znacznie mniej uniwersalny.